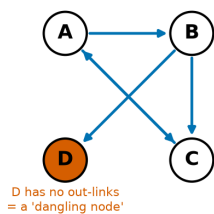


Accelerating PageRank with Dynamic Momentum Power Iteration

A beginner's guide to the whole project:
the idea, the code, the data, and every figure explained

1. A tiny web of 4 pages

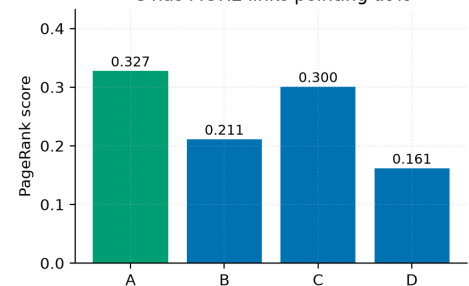


2. Its link matrix P
(column j = how page j splits its vote)

	from A	B	C	D
to A	0.00	0.00	1.00	0.00
to B	0.50	0.00	0.00	0.00
to C	0.50	0.50	0.00	0.00
to D	0.00	0.50	0.00	0.00

Each column adds to 1: a page splits its vote equally among its out-links.
Column D is all zeros - D links nowhere.

3. The answer: A wins - even though C has MORE links pointing at it



C has 2 in-links, A has only 1. But C's single out-link gives ALL of C's score to A, while A splits its score two ways. PageRank measures WHO links to you, not how many.

Course	CSE 402 — Numerical Analysis Sessional, Section C
Base paper	Austin, Pollock & Zhu (2024), <i>Dynamically accelerating the power iteration with momentum</i> , Numer. Linear Algebra Appl. 31(6), e2584
Group	Nakib Arman (2105128) · Ali Asif Khan (2105131) · Shariar Al Kabir (2105132) · Dip Saha (2105138) · MD. Mehedi Hasan Mim (2105142)
Presenter	Dip Saha
Code	7 library modules · 7 experiments · 19 unit tests · make all

How to read this guide

This guide is written for someone who has **not** yet understood the project. It starts from “what is PageRank” and ends at “why our answer came out the way it did”. Nothing is assumed except comfort with matrices and vectors.

It is deliberately different from PROJECT.md, which is the formal report and assumes you already know the theory. If you only have time for one, read this one first.

Part	What it answers
1. The problem	What is PageRank, what is an eigenvector here, why is it slow?
2. The idea	What is momentum, what does β do, how does the paper guess β ?
3. Symbols	Every letter in the code and the report, in one dictionary.
4. The data	Which graphs, where from, what the raw files look like.
5. The code	Every file, what it does, with real excerpts.
6. The figures	All 13 figures, what is plotted and what to look for.
7. The result	What we found and, in plain words, why.
8. Running it	Commands, and what to say when someone asks.

READ THIS FIRST

There are **two different things called d** in this project, and mixing them up is the single most common confusion. **d** (no subscript) is the *damping factor*, usually 0.85. **d_k** (with a subscript) is the *residual* at step k , a measure of how wrong the current answer still is. Part 3 lists both.

Part 1 — The problem, from zero

1.1 What PageRank actually computes

Imagine a tiny web of four pages that link to each other. We want a single number per page saying how important it is. PageRank's rule is circular on purpose: **a page is important if important pages link to it**, and a page splits its own importance equally among the pages it links to.

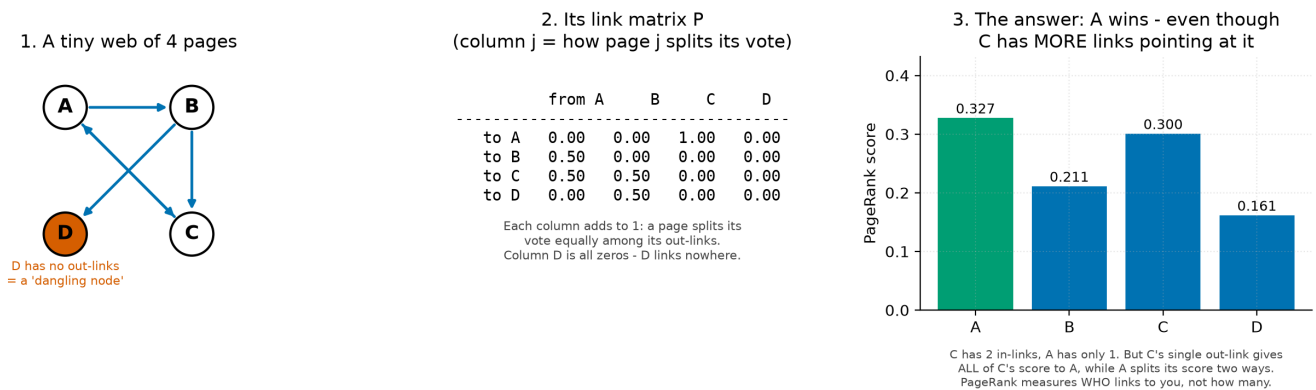


Figure A. Left: four pages and their links. Middle: the same thing written as a matrix P , where column j says how page j splits its vote. Right: the resulting scores. Notice A wins even though C has more incoming links — because C's only out-link hands all of C's score to A, while A splits its score two ways.

That last point is the whole idea of PageRank and worth repeating: it is not a popularity count. A single link from a page that links nowhere else is worth more than many links from pages that link everywhere.

The matrix P

Column j of P describes page j 's outgoing links. If page j has 3 out-links, each gets $1/3$. So **every column adds up to 1** — this is called *column-stochastic*. In the figure, column A is (0, 0.5, 0.5, 0) because A links to B and C.

Dangling nodes

Page D links nowhere, so its column is all zeros and it would leak importance out of the system. Such pages are called **dangling nodes**. The fix is to pretend a dangling page links to every page equally. Real data is full of these: our citation graph has 2,393 of them (6.9% of all papers).

The damping factor d

There is one more ingredient. A surfer who only ever follows links can get stuck in a loop. So with probability d (typically 0.85) the surfer follows a link, and with probability $1 - d$ they jump to a completely random page. That jump is called *teleportation*, and it guarantees a unique answer exists.

Putting the three pieces together gives the **Google matrix**:

$$G = d \cdot \left(P + \frac{1}{n} \mathbf{e} \mathbf{a}^T \right) + \frac{(1-d)}{n} \mathbf{e} \mathbf{e}^T$$

$\uparrow$ follow a link $\uparrow$ dangling-node fix $\uparrow$ teleport to a random page

Here $\mathbf{e}$ is a column of all ones, $\mathbf{a}$ marks which pages are dangling, and n is the number of pages. The PageRank scores are the vector $\mathbf{x}$ satisfying $G \mathbf{x} = \mathbf{x}$.

1.2 Why this is an eigenvector problem

$G \mathbf{x} = \mathbf{x}$ says: applying G to $\mathbf{x}$ gives back $\mathbf{x}$ itself. In linear algebra language, $\mathbf{x}$ is an *eigenvector* of G with *eigenvalue* 1. Because G is column-stochastic, 1 is always its largest eigenvalue, and the

matching eigenvector is exactly the PageRank vector. So **computing PageRank = finding the dominant eigenvector of G** .

NOTE

This is why the base paper is relevant at all. The paper is about finding dominant eigenvectors faster. PageRank is a dominant-eigenvector problem, on a gigantic sparse matrix. That is the “cross-domain application” in our title.

1.3 Power iteration: just multiply, over and over

The oldest method for this is embarrassingly simple. Start with any guess x_0 . Multiply by G . Normalise so it does not blow up. Repeat.

```
x = starting guess
repeat:
  x = G x          # one matrix-vector product
  x = x / ||x||    # keep it a sensible size
until x stops changing
```

Why does this work? Write your starting guess as a mixture of all the eigenvectors of G . Each multiplication by G multiplies the i -th ingredient by its eigenvalue λ_i . The largest eigenvalue is $\lambda_1 = 1$, so after k steps the ingredient for λ_2 has shrunk by $(\lambda_2/\lambda_1)^k$ relative to the winner, the one for λ_3 by even more, and so on. Everything except the dominant eigenvector dies away.

1.4 Why it is slow: the spectral gap

The speed is decided by a single number, the ratio of the two largest eigenvalues:

$r = |\lambda_2 / \lambda_1|$ ← called the spectral ratio, or the gap

Each iteration kills only a fraction r of the remaining error. If $r = 0.5$ the error halves every step and you are done in 30 steps. If $r = 0.99$ you need about 2,000 steps for the same accuracy. **The closer r is to 1, the slower everything gets.**

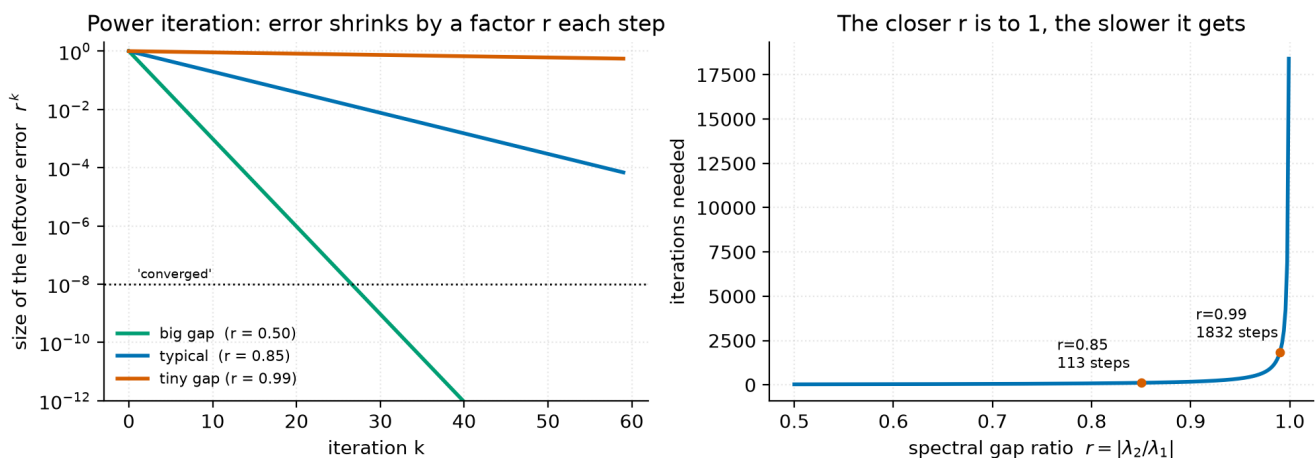


Figure B. Left: the leftover error after k steps is r^k . Right: how many steps are needed to reach 10^{-8} . The cost explodes as r approaches 1.

WHY WE CARE

For the Google matrix there is a theorem (Haveliwala & Kamvar, 2003) that $|\lambda_2| \leq d$, the damping factor. So choosing $d = 0.99$ to get a more accurate ranking directly makes the computation about 14× slower. That tension is exactly what our project set out to fix.

Part 2 — The idea: momentum

2.1 The intuition

Plain power iteration takes one step, looks around, takes another step. It has no memory. If ten steps in a row all pushed in roughly the same direction, it still takes the eleventh step as timidly as the first.

Momentum gives it memory. Think of pushing a child on a swing: you do not fight the swing, you add a small push *in the direction it is already going*. The method does the same by subtracting a multiple of the *previous* iterate:

```
plain:       $u_{k+1} = G x_k$ 
momentum:    $u_{k+1} = G x_k - (\beta / h_k) x_{k-1}$ 
               ↑
            the memory term
```

β (beta) controls how much memory. $\beta = 0$ gives back plain power iteration. Too much β and the method becomes unstable. The whole game is choosing β well.

NOTE

The extra term costs a subtraction and a scaling — no extra matrix-vector product. That matters: on our biggest graph one matrix-vector product touches 2.3 million edges, whereas the momentum term touches 281,903 numbers. This is why we count **matrix-vector products**, not seconds, when comparing methods.

2.2 The trick that makes it analysable: the augmented matrix

Here is the clever step in the theory. The momentum iteration looks like it depends on two previous vectors, which is awkward. But if you stack the current and previous iterate into one long vector, it becomes an ordinary *plain* power iteration on a bigger matrix:

$$A\beta = \begin{bmatrix} A & -\beta I \\ I & 0 \end{bmatrix} \quad \text{applied to} \quad \begin{bmatrix} x_k \\ x_{k-1} \end{bmatrix}$$

So momentum is not a new kind of algorithm at all — it is the same old power iteration on a cleverly chosen larger matrix. And the speed of a power iteration is decided by the eigenvalues of the matrix it runs on. Each eigenvalue λ of G becomes **two** eigenvalues of $A\beta$:

$$\mu_{\pm} = (\lambda \pm \sqrt{\lambda^2 - 4\beta}) / 2 \quad \leftarrow \text{equation (2.7) in the paper}$$

THE KEY FORMULA

Everything in this project follows from that one formula. If you remember one line from this guide, make it this one. Our result, the failure, and the fix are all just consequences of what μ does as λ and β change.

2.3 What β does — the “flattening” picture

Multiply the two roots together and you always get β (this is Vieta's formula: for $\mu^2 - \lambda\mu + \beta = 0$ the product of the roots is the constant term). So $|\mu_+| \times |\mu_-| = \beta$, **always**. The only question is how the two split.

If λ is **real** and small enough that $\lambda^2 < 4\beta$, the square root is imaginary, the two roots are complex conjugates, and conjugates have *equal* size. Their product is β , so each must be exactly $\sqrt{\beta}$. They cannot split.

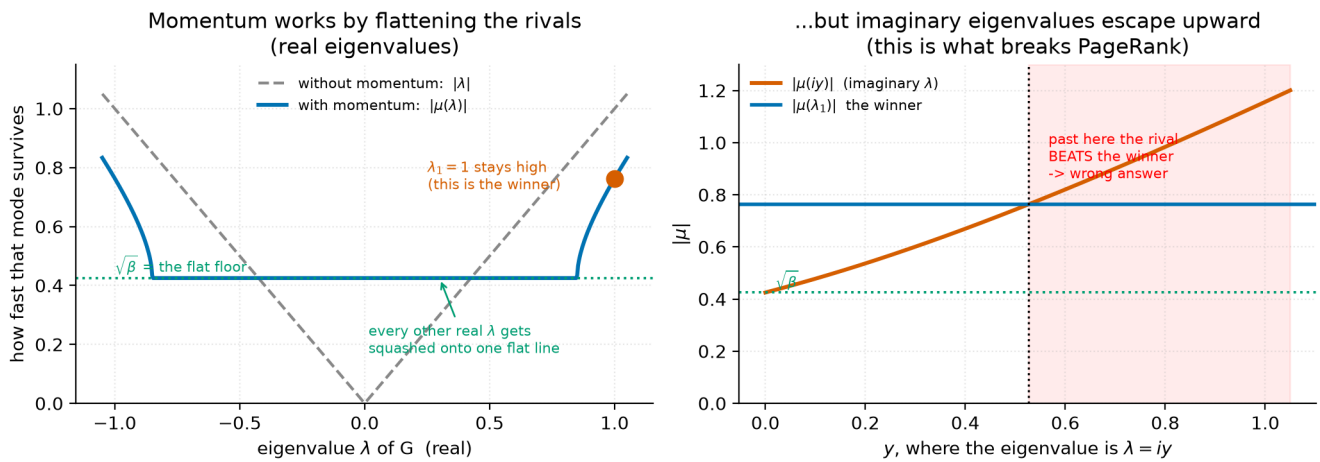


Figure C. Left: with momentum, every real eigenvalue below a threshold gets squashed onto one flat floor at $\sqrt{\beta}$, while $\lambda_1 = 1$ stays high. That gap is the acceleration. Right: an imaginary eigenvalue is not squashed — it climbs, and past the dotted line it beats λ_1 and the method converges to the wrong answer.

The left panel is the acceleration in one picture. Without momentum a rival mode at $\lambda = 0.85$ survives at strength 0.85 and only barely dies away. With momentum it is crushed to 0.43, while the winner stays at 0.76. The bigger that gap, the faster you finish.

Choosing $\beta = \lambda_2^2/4$ puts the flat floor exactly at the highest rival, which is the best you can do. That is the paper's **optimal β** . The resulting speed is

$$\rho(r) = r / (1 + \sqrt{1 - r^2}) \quad \text{versus } r \text{ without momentum}$$

$r = \lambda_2/\lambda_1 $	$\rho(r)$ with momentum	predicted speed-up
0.85	0.5567	3.60×
0.90	0.6268	4.43×
0.95	0.7239	6.30×
0.99	0.8676	14.13×
0.999	0.9562	44.72×

The worse the problem, the more momentum helps. That is exactly why PageRank looked like a perfect target.

2.4 The paper's contribution: guessing β without knowing λ_2

There is a catch. $\beta = \lambda_2^2/4$ needs λ_2 , and finding λ_2 is about as hard as the original problem. Earlier methods either assumed you knew it or spent extra matrix-vector products estimating it.

The paper's idea is to **watch how fast the method is actually converging and work backwards**. If the residual shrinks by a factor ρ each step, and theory says the best possible shrink factor is $\rho = r/(1+\sqrt{1-r^2})$, then you can invert that relation to recover r :

$$\begin{aligned} \rho_k &= \min(d_{k+1} / d_k, 1) && \leftarrow \text{the observed shrink factor} \\ r_{k+1} &= 2\rho_k / (1 + \rho_k^2) && \leftarrow \text{invert the formula to get } r \\ \beta_k &= v_k^2 \cdot r_k^2 / 4 && \leftarrow \text{and hence } \beta \end{aligned}$$

This costs nothing: d_k and v_k are already computed to check convergence. That is **Algorithm 3.1**, the paper's method, and it is genuinely elegant — it tunes itself from information it was collecting anyway.

FORESHADOWING

Hold on to this, because it is where the project turns. The self-tuning rule assumes the residual behaves the way a *real* spectrum makes it behave. When we apply it to PageRank, that assumption breaks, and the rule turns from an asset into the cause of failure. Part 7 tells that story.

Part 3 — Every symbol, in one place

This is the dictionary to keep open while reading the code or the report. The names in the code deliberately match the paper's notation so the two can be read side by side.

3.1 The iteration variables

Symbol	In code	What it is
x_k	x	The current guess at the answer, scaled to length 1. After convergence this <i>is</i> the PageRank vector.
x_{k-1}	x_prev	The previous guess. Only momentum needs this — it is the “memory”.
u_{k+1}	u	The new vector before scaling, i.e. $G x_k$ minus the momentum term.
v_{k+1}	v_next	The result of the matrix-vector product $G x_k$. Computed once per iteration — this is the expensive line.
h_k	h	The length of u_k , used to rescale. Prevents numbers growing or shrinking out of range.
v_k	nu	The Rayleigh quotient $\langle G x, x \rangle$ — the current estimate of the eigenvalue λ_1 . For PageRank it should approach 1.
d_k	d	The residual , $\ G x - v x\ $. How wrong we still are. We stop when this drops below 10^{-12} .
ρ_k	rho	The observed shrink factor d_{k+1}/d_k . How fast the residual is actually falling.
r_k	r	The <i>estimate</i> of the true spectral ratio, obtained by inverting the rate formula from ρ_k .
β_k	beta	The momentum parameter. 0 = no momentum. Too large = instability.

COMMON CONFUSION

The two d's. d without a subscript is the damping factor (0.85). d_k with a subscript is the residual at step k . They are unrelated. The paper uses both and so do we, because changing the notation would make the code harder to check against the paper.

3.2 The matrix and spectral quantities

Symbol	In code	What it is
G	GoogleMatrix	The Google matrix. Never built explicitly — it is dense and would need 10^{11} numbers for our biggest graph.
P	G.P	The sparse link matrix: $P[i,j] = 1/\text{outdeg}(j)$ if page j links to page i .
d	G.d	The damping factor, 0.85 by default.
n	G.n	Number of pages/nodes.
a	G.dangling	Boolean mask: true for pages with no out-links.
e	—	A column of all ones. Never stored; it appears only in formulas.
λ_1	—	Largest eigenvalue of G . Always exactly 1 for a Google matrix.
λ_2	lambda2_measured	Second largest. Bounded by d ; we measure whether the bound is tight.
r	r	The spectral ratio $ \lambda_2/\lambda_1 $. For PageRank this equals d when the bound is tight.
$\mu_{\pm}$	mu_magnitude()	Eigenvalues of the augmented matrix, from equation (2.7). These decide whether momentum helps or destroys.
$A\beta$	—	The augmented matrix. An analysis device only; never built in code.

3.3 The five methods we compare

Name in code	What it is
power	Algorithm 1.1 — plain power iteration. The baseline everything is measured against.
static_d	Algorithm 1.2 with $\beta = d^2/4$ — the paper's optimal fixed β , which for PageRank we get for free because $\lambda_1 = 1$ and $\lambda_2 \leq d$.
dynamic_paper	Algorithm 3.1 exactly as printed in the paper.
dynamic_safe	Our extension: Algorithm 3.1 plus the safeguards of Part 7.
dynamic_guard	The same plus an extra-cautious rule that retreats to plain power iteration as soon as momentum stops paying.

Part 4 — The data

4.1 Where it comes from

Two sources, for two different jobs.

SNAP — Stanford Large Network Dataset Collection

Real graphs, downloaded from snap.stanford.edu/data. These are the PageRank problems. They are cached in `data/` on first use, so you only download once.

SuiteSparse Matrix Collection

The abstract benchmark matrices the base paper itself used, fetched via the `ssgetpy` package. We use these to *reproduce the paper* before trusting our code on anything new.

4.2 Every dataset we use

Dataset	Nodes	Edges	Kind	What we use it for
wiki-Vote	7,115	103,689	social	Small enough to debug on, and the one graph where $ \lambda_2 $ is far below d — a useful counter-example.
cit-HepPh	34,546	421,578	citation	Main citation case. arXiv high-energy-physics papers citing each other. This is the dataset named in our proposal.
web-Stanford	281,903	2,312,497	web	Main web case. Pages on stanford.edu and the links between them.
ca-HepPh	12,008	118,521	collaboration	Symmetric control. Undirected co-authorship: two authors are joined if they wrote a paper together.
cit-HepTh	27,770	352,807	citation	Available, unused.
web-NotreDame	325,729	1,497,134	web	Available, unused.
web-Google	875,713	5,105,039	web	Supported; the practical ceiling for a laptop.
Kuu, Muu	7,102	—	SuiteSparse	Paper's test suite 1. Both symmetric positive definite, $r \approx 0.998$.
ash292, bcspwr06	292 / 1,454	—	SuiteSparse	Paper's test suite 2. Symmetric but indefinite.

4.3 What the raw file actually looks like

A SNAP file is a gzipped text file of number pairs, with a comment header. Nothing exotic:

`data/cit-HepPh.txt.gz` (decompressed)

```
# Directed graph: cit-HepPh.txt
# Paper citation network of Arxiv High Energy Physics category
# Nodes: 34546 Edges: 421578
# FromNodeId    ToNodeId
9907233    9301253
9907233    9504304
9907233    9505235
...
```

Each line means “paper 9907233 cites paper 9301253”. Two things make this harder than it looks:

- **The IDs are not 0,1,2,...** They are arXiv identifiers with big gaps. We renumber them to $0..n-1$ with `np.unique(..., return_inverse=True)` and keep the original IDs so results can be reported in the source's own numbering.

- **The dumps contain self-loops and duplicate edges.** Both are artefacts; a paper citing itself is not a real link. We drop both.

4.4 From a file to a ranking

How the project turns a downloaded file into a ranking

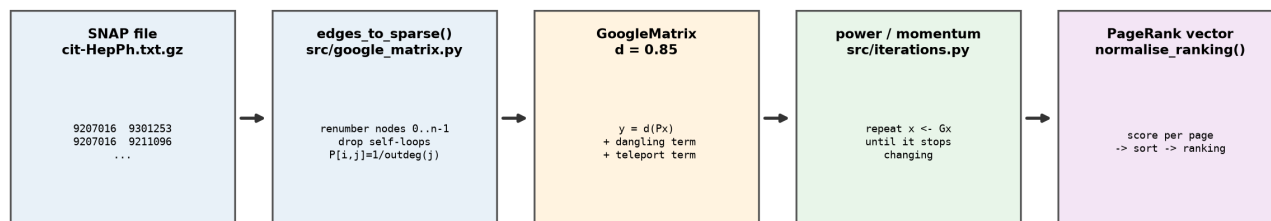


Figure D. The five stages every experiment goes through. Only the fourth stage differs between the methods we compare — everything else is shared, which is what makes the comparison fair.

4.5 Why we never build G

For web-Stanford, G is $281,903 \times 281,903$. Storing it densely would need about 636 **gigabytes**. But G is mostly the sparse matrix P plus two rank-one corrections, so we apply it as a formula instead:

```

y = d · (P @ x)                # sparse, ~2.3M multiply-adds
+ (d/n) · (a · x) · e          # dangling-node fix, one dot product
+ ((1-d)/n) · (e · x) · e      # teleportation, one dot product
  
```

That is one sparse matrix-vector product plus two dot products, and it runs in about 3 milliseconds. We count it as **one matrix-vector product**.

TRAP 1

Most PageRank code simplifies the last two terms to a single constant, because plain power iteration keeps the entries of x summing to 1. **Momentum breaks that.** Its iterates are scaled to length 1 in the 2-norm and can contain *negative* entries, so the sum is neither 1 nor even positive. Using the shortcut gives a silently wrong operator and a silently wrong ranking. This cost us real debugging time and is now locked down by a unit test that checks the identity on negative and zero vectors.

Part 5 — The code, file by file

About 3,600 lines, laid out so that each file has one job.

```
src/                                the library
  iterations.py                    (698)  the three algorithms + our safeguards
  google_matrix.py                (236)  the Google matrix as a formula
  datasets.py                     (241)  downloading and caching graphs
  spectrum.py                     (168)  the  $\mu(\lambda)$  analysis of Part 7
  metrics.py                      (115)  ranking quality: precision@k, Kendall tau
  pagerank_methods.py             (76)   the five configurations, defined once
  plotting.py                     (66)   one shared figure style

experiments/                       one file per phase, each runnable alone
  exp1_reproduce.py                 reproduce the paper          (a PASS/FAIL gate)
  exp2_pagerank.py                validate G, then benchmark  (a PASS/FAIL gate)
  exp3_damping.py                 sweep the damping factor d
  exp4_ranking.py                 ranking quality vs residual
  exp5_spectrum.py                why  $\beta = d^2/4$  fails
  exp6_symmetric.py               the symmetric control
  exp7_normality.py               the single-variable proof

tests/                             19 unit tests
results/                           CSV output + run.log
figures/                           PDF for LaTeX, PNG for slides
```

5.1 src/iterations.py — the three algorithms

The most important file, and the one design decision worth understanding: **this file knows nothing about PageRank**. Every function takes an arbitrary linear operator and only ever computes $A @ x$.

That is why the same code runs on the paper's SuiteSparse matrices and on our Google matrix with no changes — and it is why, when the PageRank results came out negative, we could *prove* the solver was correct rather than argue about it.

Algorithm 1.1, the whole loop

```
def power_iteration(A, v0=None, tol=1e-12, max_iter=2000, ...):
    matvec = _as_matvec(A)
    ...
    for k in range(1, max_iter + 1):
        h = np.linalg.norm(v_next)
        x = v_next / h
        v_next = matvec(x)          # the single matvec
        matvecs += 1
        nu = float(v_next @ x)      # Rayleigh quotient
        d = float(np.linalg.norm(v_next - nu * x)) # residual
        if d < tol:
            converged = True
            break
```

The momentum version adds exactly one line before the rescale, and the dynamic version adds three more to update β :

```
# Algorithm 1.2 - static momentum
u = v_next - (beta / h) * x_prev

# Algorithm 3.1 - dynamic momentum: beta is recomputed every step
beta = nu * nu * r * r / 4.0
u = v_next - (beta / h) * x_prev
...
rho = min(d / d_prev, 1.0)
r = 2.0 * rho / (1.0 + rho * rho)
```

Every routine returns an `IterationResult` carrying the full history — residual per step, β per step, matvec count, wall-clock, and flags for whether it converged or diverged. The experiment scripts do nothing but call these and tabulate.

5.2 src/google_matrix.py — G without building G

```
class GoogleMatrix:
    def matvec(self, x):
        self.count += 1
        x = np.asarray(x).ravel()
        y = self.d * (self.P @ x)
        dangling_mass = x[self.dangling].sum()
        total_mass = x.sum()
        y += (self.d * dangling_mass + (1.0 - self.d) * total_mass) / self.n
        return y
```

Three details worth noting. The count makes every experiment's matvec tally automatic rather than hand-maintained. `with_damping(d)` re-uses the sparse structure so sweeping `d` does not rebuild the matrix. And the input dtype is preserved, because the eigenvalue solver probes the operator with *complex* vectors — quietly casting to float would corrupt the analysis in Part 7.

5.3 The supporting modules

File	What it gives you
<code>datasets.py</code>	Downloads and caches SNAP and SuiteSparse data. Also holds a synthetic graph generator with power-law out-degrees and dangling nodes, so the whole project still runs with no internet (<code>--offline</code>).
<code>metrics.py</code>	Ranking-quality measures. The paper only ever looks at the residual; these ask the question a search engine actually cares about — is the <i>order</i> right? precision@k, Kendall tau, ℓ_1 error.
<code>spectrum.py</code>	The analysis machinery for Part 7: $\mu(\lambda)$ from equation (2.7), the true convergence rate as a function of β , and the stability limit. It samples eigenvalues by largest magnitude <i>and</i> largest/smallest imaginary part — for a reason explained in Part 7.
<code>pagerank_methods.py</code>	Defines the five configurations once, so every experiment compares identical settings.
<code>plotting.py</code>	One colour-blind-safe palette and one style, so every figure in the report matches.

5.4 The gates

Two experiments print an explicit **PASS/FAIL** and exit non-zero on failure.

- **Phase 1 gate** (exp1): does our code reproduce the paper on the paper's own matrices? If not, nothing downstream means anything.
- **Phase 2 gate** (exp2): is our Google matrix correct? Checked against NetworkX *and* against its own fixed point $\|Gx - x\|_1 < 10^{-12}$, which needs no other library to be trusted.

Part 6 — Every figure explained

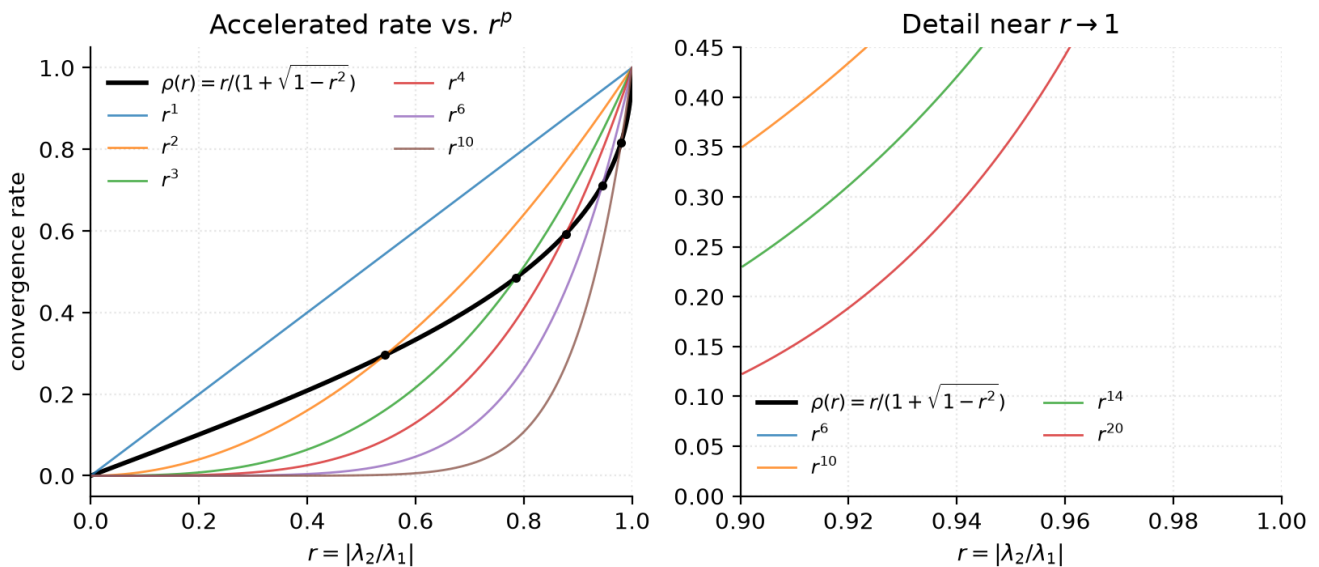
Thirteen figures. For each one: what is plotted, and what you are supposed to notice. All exist as both PDF (for LaTeX) and PNG (for slides) in figures/.

NOTE

Several figures are wide and appear small here. Open the matching .pdf in figures/ to zoom in — they are vector graphics, so they stay sharp at any magnification.

6.1 fig1_rate_comparison — how much momentum could help

Reproduction of Figure 1 (Austin, Pollock & Zhu 2024)

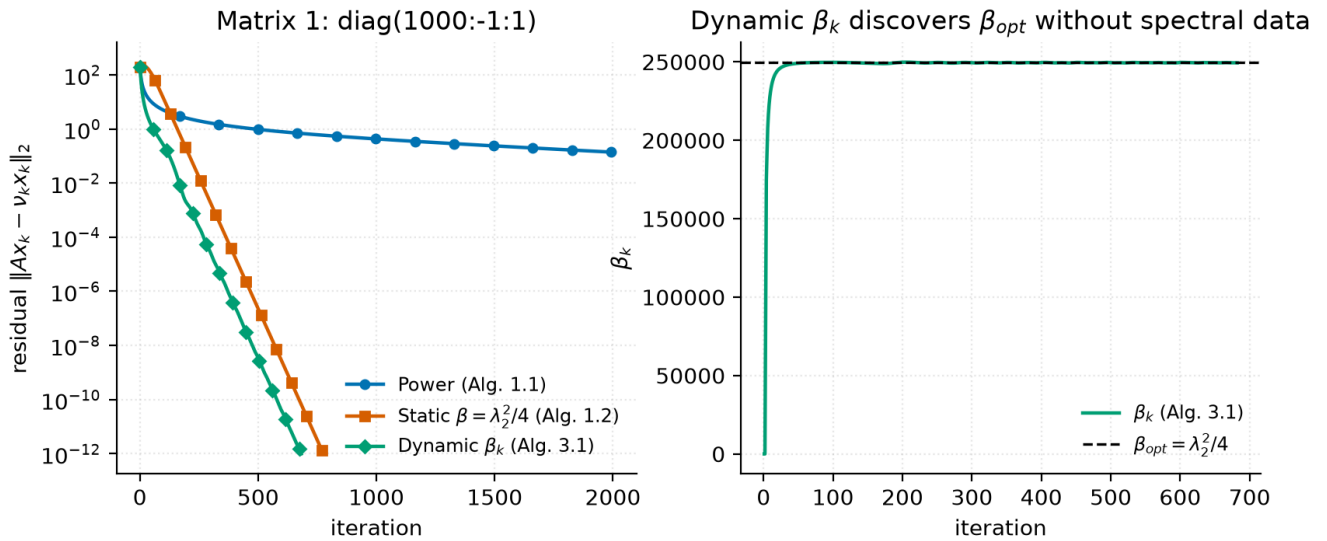


Reproduction of Figure 1 of the paper. The black curve is the accelerated rate $\rho(r)$; the thin curves are r^p for various p .

Open figures/fig1_rate_comparison.pdf to zoom in.

Read it as: momentum turns a rate of r into a rate of $\rho(r)$. The black curve sitting below r^6 for $r > 0.945$ means that in that regime one momentum step is worth about six plain steps. **Look for:** the curve dropping further below the thin lines as $r \rightarrow 1$. This is pure theory — no data yet.

6.2 fig3_matrix1 — the reproduction, and β tuning itself



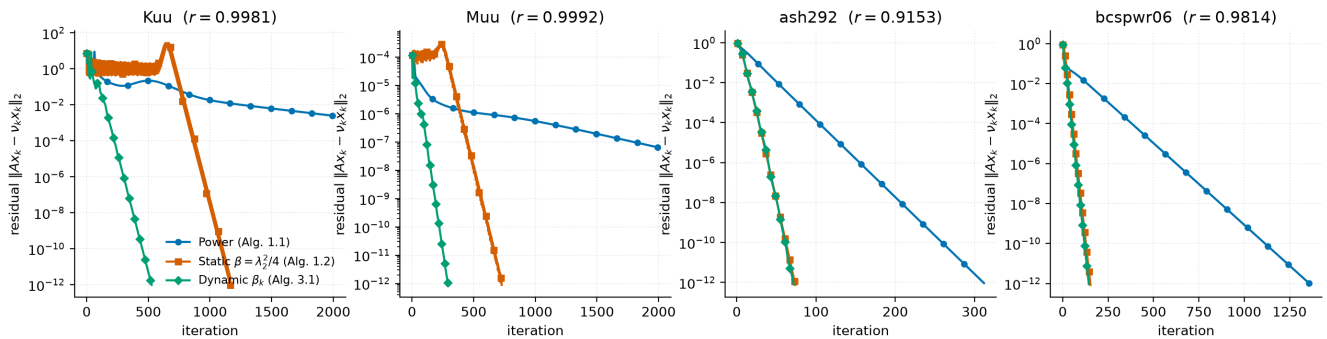
Left, look for: the green curve (dynamic) plunging while the blue (plain power) barely moves. 683 matvecs versus 27,619 — a 40× speed-up, against 44.7× predicted.

Right, look for: the green line settling onto the black dashed line. That dashed line is $\lambda_2^2/4$, which the algorithm was *never told*. It found it from residual ratios alone. This panel is the paper's central claim, working.

Left, look for: the green curve (dynamic) plunging while the blue (plain power) barely moves. 683 matvecs versus 27,619 — a 40× speed-up, against 44.7× predicted.

Right, look for: the green line settling onto the black dashed line. That dashed line is $\lambda_2^2/4$, which the algorithm was *never told*. It found it from residual ratios alone. This panel is the paper's central claim, working.

6.3 exp1_suitesparse — the same on the paper's real matrices

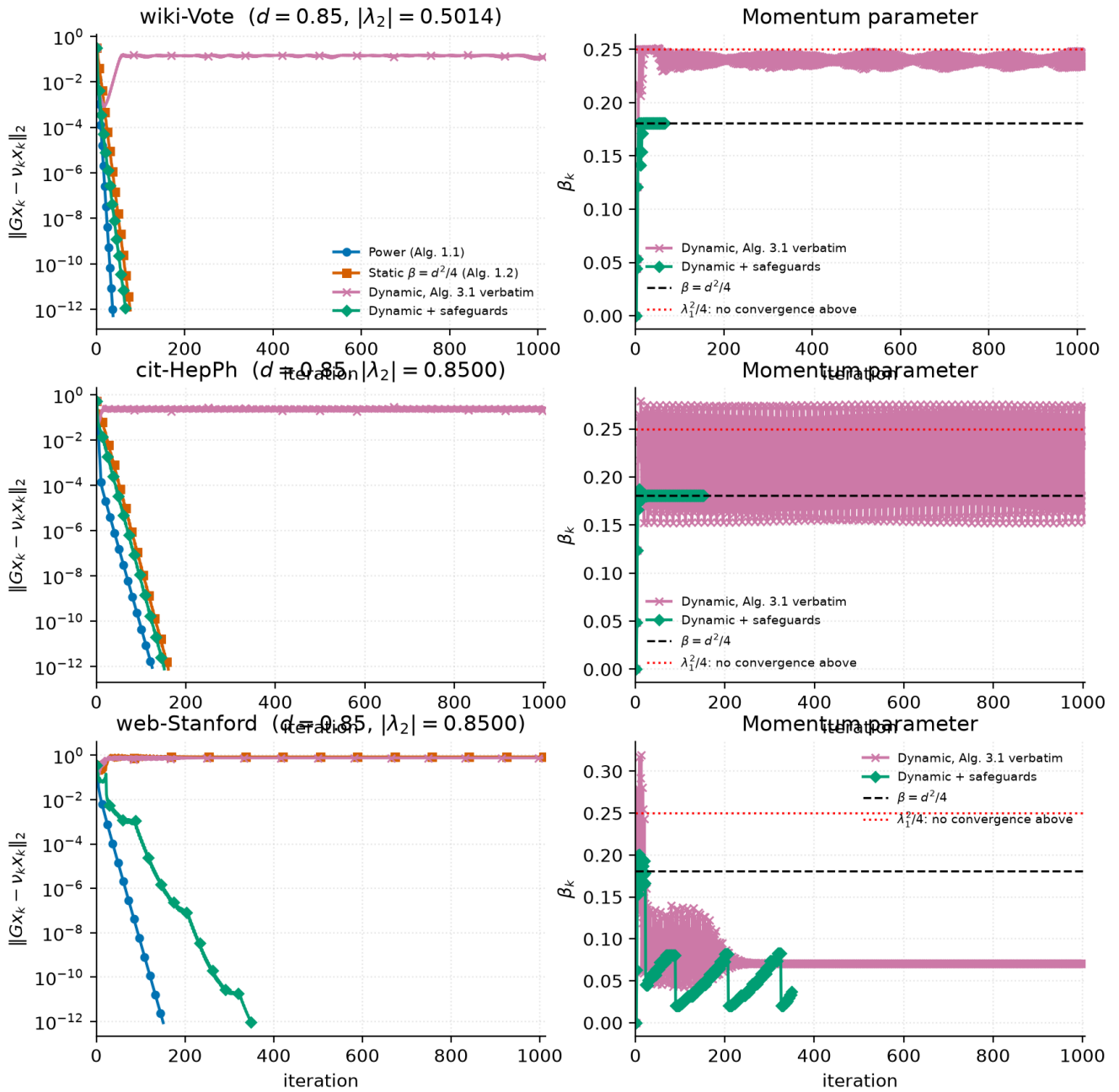


Kuu, Muu, ash292 and bccspwr06 from the SuiteSparse collection.

Open figures/exp1_suitesparse.pdf to zoom in.

Look for: dynamic (green) at or below static (orange) on all four. Our measured spectral ratios match the paper's published values to four decimals, which is independent evidence we built the right matrices.

6.4 exp2_pagerank_convergence — the first PageRank result



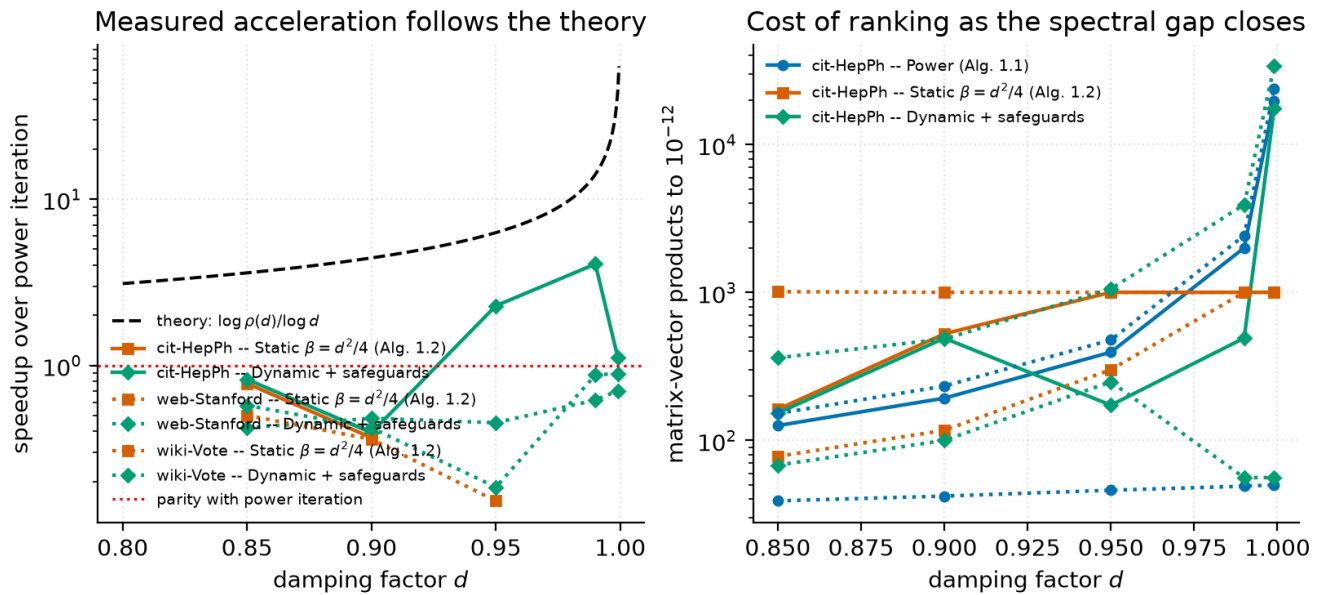
One row per graph. Left column: residual against iteration. Right column: the β chosen, against $d^2/4$ and against the $\frac{1}{4}$ barrier.

Open figures/exp2_pagerank_convergence.pdf to zoom in.

Left, look for: the pink curve (Algorithm 3.1 as published) *flattening out and turning upward* instead of falling. It is not slow — it is diverging. Plain power iteration (blue) wins on all three graphs.

Right, look for: the pink β line climbing to the red dotted line at 0.25. Section 2 of the paper proves convergence is impossible at or above $\beta = \lambda_1^2/4 = \frac{1}{4}$. The algorithm drives itself there. This panel is the failure, caught in the act.

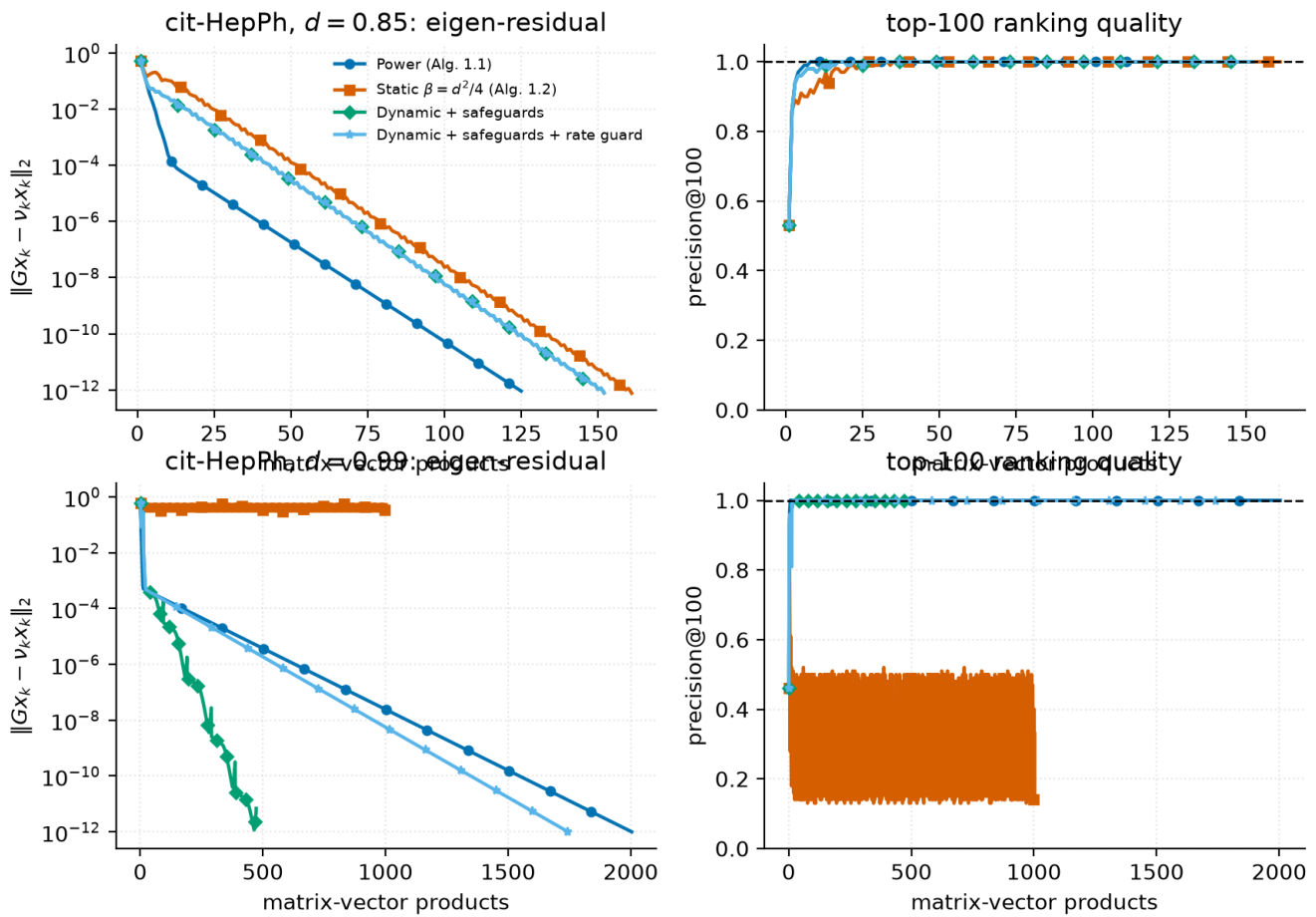
6.5 exp3_damping_sweep — measured against predicted



Left: speed-up against the damping factor, with the theoretical curve dashed. Right: cost as the spectral gap closes. Open figures/exp3_damping_sweep.pdf to zoom in.

Look for: the gap between the dashed theory curve and the measured points, and the red parity line at 1.0. Almost everything sits *below* parity — slower than plain power iteration. The exception is cit-HepPh at $d = 0.95$ and 0.99 , where our safeguarded version does reach $2.3\times$ and $4.1\times$.

6.6 exp4_ranking_quality — the practical punchline



Left column: the eigenvalue residual. Right column: precision@100, i.e. how much of the true top-100 we have found.

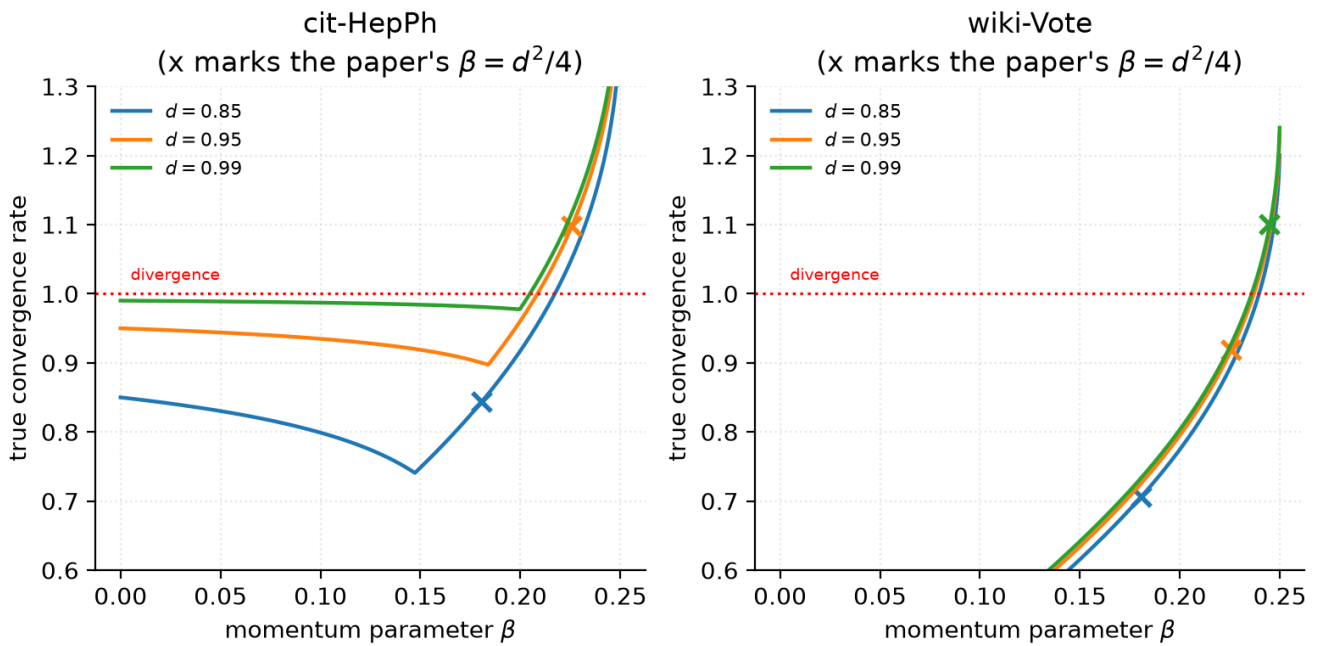
Open figures/exp4_ranking_quality.pdf to zoom in.

Look for: the right-hand curves hitting 1.0 almost immediately, while the left-hand curves keep falling for hundreds more steps. At $d = 0.99$ the top-100 ranking is correct after **7** matrix-vector products; driving the residual to 10^{-12} takes **2,005**.

WHY IT MATTERS

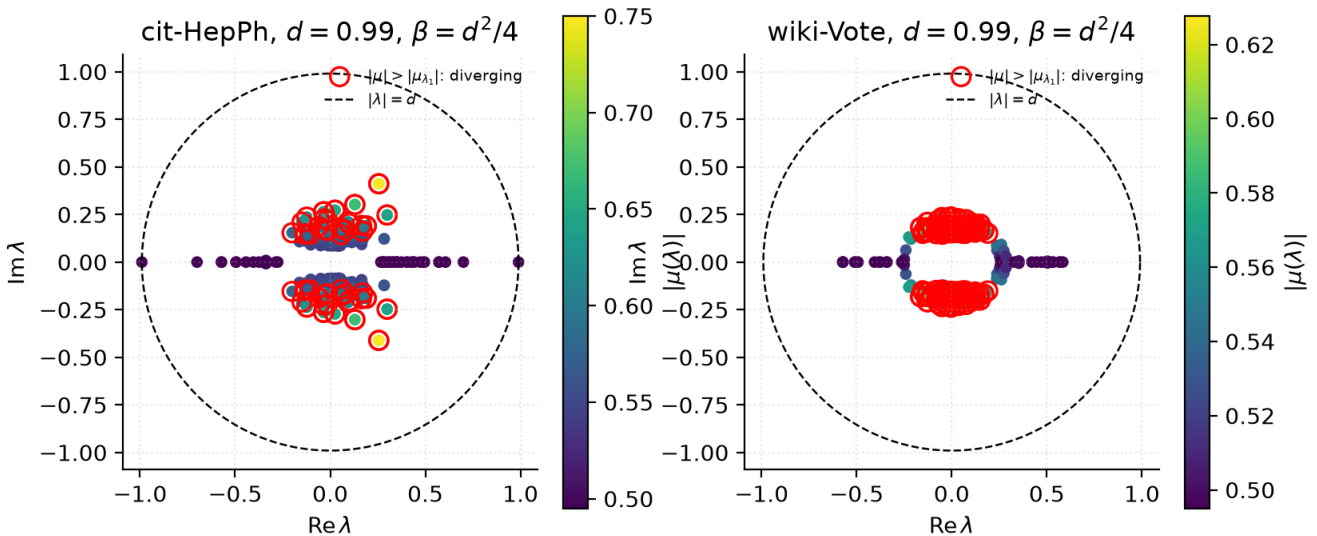
This is arguably the most useful finding in the project. For PageRank, accelerating the tail of the residual optimises a quantity nobody reads. Whatever the momentum debate, *the ranking was already right*.

6.7 exp5_rate_vs_beta — the paper's β is out of bounds



Look for: the red dotted line at rate = 1. Anything above it diverges. At $d = 0.95$ and 0.99 the x sits *above* that line. The paper's optimal parameter is not merely sub-optimal for PageRank — it is outside the stability region.

6.8 exp5_spectrum_plane — the smoking gun

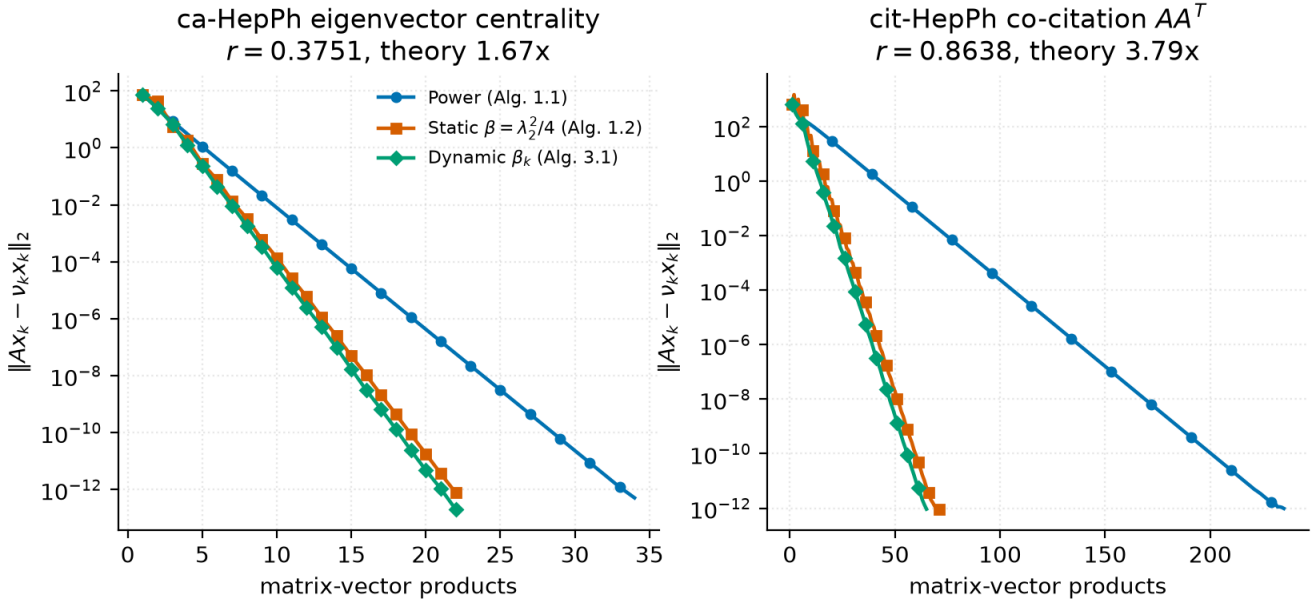


Open figures/exp5_spectrum_plane.pdf to zoom in.

Look for: where the red circles are. They are not near the outer dashed circle where the big eigenvalues live. They are clustered near the centre, just off the horizontal axis — small eigenvalues with a large imaginary part.

That is the counter-intuitive heart of the project. The modes that destroy the method are the ones a normal eigenvalue solver would never even return, because it returns the *largest* ones. If you show one figure in the presentation, show this one.

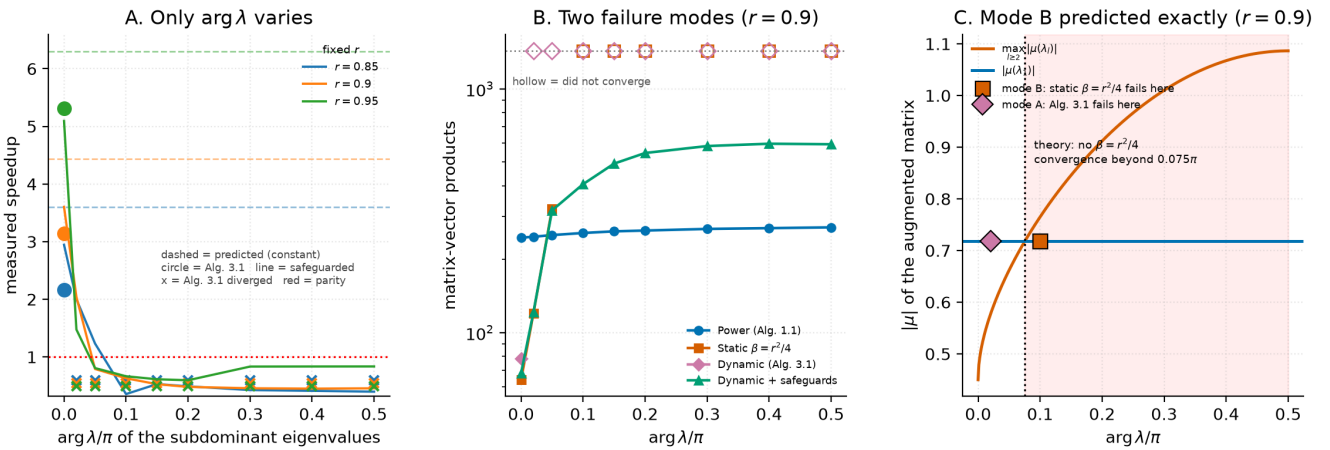
6.9 exp6_symmetric_control — proof the code is fine



The same three solvers on two symmetric ranking problems.
Open figures/exp6_symmetric_control.pdf to zoom in.

Look for: momentum working normally here — $3.58\times$ against $3.79\times$ predicted on the right-hand panel. That panel uses the **same cit-HepPh data** as the failing PageRank run, just made symmetric (co-citation: how many papers cite both i and j). Same data, symmetric instead of directed, and the acceleration appears.

6.10 exp7_normality — the controlled proof



Three panels holding everything fixed except one variable.
Open figures/exp7_normality.pdf to zoom in.

A sharp examiner can object to 6.9: the co-citation control changes the symmetry *and* the matrix *and* r all at once. So how do we know which one matters? This figure answers that by changing exactly one thing.

- **Panel A:** $\lambda_1 = 1$ and $|\lambda_2| = r$ are held fixed, so the dashed predicted speed-up is the same on every point. Only the *angle* of the subdominant eigenvalues is rotated. The circles (Algorithm 3.1) exist only at angle 0; everywhere else it diverged.
- **Panel B:** the blue power-iteration line is almost flat — the problems are equally hard throughout, so nothing can be blamed on difficulty. Meanwhile the momentum methods collapse.

- **Panel C:** the orange curve is the strongest rival mode, the blue line is the winner. Where they cross, theory says the static method must fail — and the orange square (measured) sits exactly there. Algorithm 3.1 (pink diamond) fails even *earlier*, for a different reason.

THE PROOF

Momentum goes from 3.14x to divergence after as little as a **3.6° rotation** of the spectrum, with every other quantity pinned. One variable in, one outcome out. This is the strongest evidence in the project and takes 20 seconds to re-run.

6.11 The teaching figures

Figures A-D earlier in this guide ([figures/explain/](#)) were made for this document, not for the report: the tiny four-page web, the spectral-gap cost curves, the momentum “flattening” picture, and the pipeline diagram. They are the ones to use if you need to explain the project to someone in two minutes.

Part 7 — What we found, and why

7.1 The short version

The method does not make PageRank faster. Plain power iteration wins on every real graph we tried. Algorithm 3.1 exactly as published *diverges* on all of them.

That is a negative result, but it is not a failed project, for three reasons: we can prove our implementation is correct, we can explain precisely why the method fails, and we can fix it. Those three things are the project.

7.2 Failure mode A: the self-tuning rule turns on itself

Recall from Part 2 that Algorithm 3.1 estimates r by watching the residual shrink. The formula includes a safety clamp, $\min(\rho, 1)$, for the case where the residual *grows*. On a symmetric matrix the residual falls monotonically and that clamp never fires.

The Google matrix is not symmetric. Its residual oscillates. The moment one step has $d_{k+1} > d_k$, the clamp gives $\rho = 1$, so

$$r = 2 \cdot 1 / (1 + 1^2) = 1 \quad \text{and therefore} \\ \beta = v^2 \cdot 1^2 / 4 \rightarrow \frac{1}{4} \quad \leftarrow \text{exactly the value where convergence stops}$$

Once there, the residual grows, ρ stays pinned at 1, and the method never recovers. On wiki-Vote the residual bottoms out near 10^{-3} around iteration 20 and climbs back to 0.15 — **2,000 iterations end further from the answer than 20 did.**

7.3 Failure mode B: complex eigenvalues (the real cause)

Bounding r fixes mode A but not this. Go back to the key formula and the Vieta observation from Part 2: the two roots always multiply to β .

- If λ is **real**, the roots are complex conjugates. Conjugates have equal size. Their product is β , so each is exactly $\sqrt{\beta}$. **They cannot split.**
- If λ is **complex**, they are not conjugates. They split — one grows, the other shrinks to keep the product at β .

For a purely imaginary $\lambda = iy$ the larger root is $(|y| + \sqrt{y^2 + 4\beta})/2$, which grows without bound in $|y|$. Concretely, at $d = 0.99$ on wiki-Vote:

eigenvalue λ	$ \lambda $	$ \mu $ after momentum	
1 (the winner)	1.000	0.5705	what we want to find
0.90 (real)	0.900	0.4950	squashed — harmless
0.23 (real)	0.230	0.4950	squashed — harmless
0.0102 + 0.2312i	0.231	0.6239	beats the winner

An eigenvalue of size **0.23** — utterly harmless to plain power iteration — overtakes the dominant mode and the iteration converges to the wrong eigenvector. On wiki-Vote at $d = 0.99$, 126 of 179 sampled modes do this.

7.4 Where the complex eigenvalues come from: direction

directed 4-cycle $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ eigenvalues: $+1, -1, +i, -i$ $\leftarrow$ complex
the same cycle undirected eigenvalues: $+2, -2, 0, 0$ $\leftarrow$ all real

Directed cycles produce roots of unity. Real web and citation graphs are full of directed cycles of every length, so the spectrum of G spreads off the real axis — we measure $|\operatorname{Im} \lambda|$ up to 0.41 on cit-HepPh. Undirected graphs give symmetric matrices, whose eigenvalues are real by theorem.

That is the whole causal chain:

directed graph $\rightarrow$ non-symmetric $G \rightarrow$ complex eigenvalues
 $\rightarrow$ the two μ roots split $\rightarrow$ a tail mode overtakes $\lambda_1 \rightarrow$ divergence

7.5 Our fix

The safeguarded variant uses only information PageRank gives away for free, plus feedback from the residual:

Safeguard	Uses	Fixes
Cap β below $\lambda_1^2/4$	$\lambda_1 = 1$ exactly, by stochasticity	Keeps β strictly inside the convergence region.
Bound $r \leq d$	the theorem $ \lambda_2 \leq d$	Kills failure mode A.
Backtracking	nothing — pure residual feedback	Handles mode B, which no advance bound can prevent.
Slow growth	nothing	Retreat alone overshoots; alternating shrink and grow keeps β just below the stability edge.
Backtrack budget	nothing	Turns it into a guarantee: worst case is power iteration plus 60 matvecs.

It converges on every graph and damping factor we tested, and on 27 of 27 matrices in the controlled sweep, reaching 4.08x on cit-HepPh at $d = 0.99$ where the published algorithm diverges.

7.6 Honest limitations

- The spectral rates come from a *sample* of the spectrum, so they are lower bounds. A sampled rate above 1 proves divergence; a sampled rate below 1 does not prove convergence. The “achievable speed-up” numbers are therefore optimistic, and exp5 measures its own predictions and prints SAMPLE INCOMPLETE where they do not hold up.
- Three real graphs, one per family. The controlled sweep in exp7 compensates by establishing the mechanism on a synthetic family instead of relying on the graph sample.
- The safeguard's constants were tuned on these graphs. The *guarantee* does not depend on them; the *acceleration* does.
- We did not implement DMPower, a competing method the paper also compares against. It would not change any conclusion, since the winner throughout is plain power iteration.

Part 8 — Running it, and answering questions

8.1 Commands

```
make setup      # create .venv and install pinned dependencies
make test       # 19 unit tests, about 1 second
make all        # every experiment; ~15 min, downloads ~90 MB the first time
make quick      # offline subset on synthetic graphs, no network needed

make phase1 ... make phase7      # one phase at a time

# every experiment takes flags:
./venv/bin/python experiments/exp2_pagerank.py --graphs wiki-Vote --offline
```

8.2 What to check if you only have a minute

```
make test          → 19 passed
make phase1 → Phase 1 gate: PASS -- proceed to Phase 2
make phase2 → Phase 2 gate: PASS
make phase7 → 3/3 exact agreement ... Non-normality is the cause.
```

8.3 Likely questions, and the honest answers

Question	Answer
"Your proposal promised to measure how much <i>faster</i> it gets. It got slower."	The promise was to <i>measure</i> , and we did — on more graphs and more damping factors than proposed. The measured answer is that it does not get faster, and we can show exactly why.
"How do you know your code is not just buggy?"	Three ways. We reproduce the paper's own benchmarks at 40.4× versus 44.7× predicted. The symmetric control on the same data gives 3.58× versus 3.79×. And 19 unit tests, including a Google matrix checked against NetworkX <i>and</i> against its own fixed point.
"Maybe the symmetric control differs for some other reason."	That is exactly why exp7 exists. It pins λ_1 and $ \lambda_2 $ and rotates only the angle of the spectrum. Power iteration varies 8–13% across the whole sweep; momentum goes from 3.14× to divergence after as little as 3.6°.
"Is the result specific to your choice of β ?"	No. exp5 scans <i>all</i> β and reports the best achievable rate, then verifies it empirically. Even the best β gives only 1.26–1.61× measured.
"So the whole thing was pointless?"	The opposite. We found the method's hidden assumption (a real spectrum), showed it fails on an important real problem, explained the mechanism quantitatively, fixed it, and found that for ranking the residual was the wrong thing to optimise anyway.

8.4 The thirty-second summary

SAY THIS

The paper's method accelerates the power iteration by reshaping the matrix spectrum so every rival mode gets crushed onto a single small circle. That reshaping only works if the eigenvalues are **real**. The Google matrix is built from a **directed** graph, so its eigenvalues are complex, and complex ones escape the circle instead of being crushed — some of them overtaking the very mode we are trying to find. We prove this with a controlled experiment, fix it with safeguards that use only facts PageRank already gives us, and show that for ranking purposes the answer was correct after 7 iterations anyway.